FIAP ON

DJANGO WEB FRAMEWORK

FORMULÁRIOS COM DJANGO

ANDREY MASIERO

5

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 5.1 – Usuários para autenticação.....5
Figura 5.2 – Cadastro de formulário do site administrativo 5
Figura 5.3 – Opções de criação do perfil.....6
Figura 5.4 – Tela de login.....12
Figura 5.5 – Tela de cadastro do usuário.....12
Figura 5.6 – Erro de CSRF.....14
Figura 5.7 – Tabela de usuários cadastrados no sistema administrativo 16
Figura 5.8 – Tabela de usuários cadastrados no sistema administrativo 21

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 5.1 – Alterando ExibeLivro para permitir edição rápida 7
Código-fonte 5.2 – Rotas de Usuário 8
Código-fonte 5.3 – Estrutura principal do arquivo views.py do app usuarios 8
Código-fonte 5.4 – Vínculo das urls no aplicativo principal 8
Código-fonte 5.5 – Template do formulário de cadastro 10
Código-fonte 5.6 – Template do formulário de login.....11
Código-fonte 5.7 – Renderização das telas.....12
Código-fonte 5.8 – Atualização do partial de cabeçalho 13
Código-fonte 5.9 – Recebendo a chamada do formulário 14
Código-fonte 5.10 – Salvando o usuário 16
Código-fonte 5.11 – Método login 17
Código-fonte 5.12 – Melhoria na partial _cabecalho.html 18
Código-fonte 5.13 – Método logout 18
Código-fonte 5.14 – Validação de autenticação em uma rota.....19
Código-fonte 5.15 – Recebendo a chamada do formulário 19
Código-fonte 5.16 – Recebendo a chamada do formulário 20
Código-fonte 5.17 – Método cadastro atualizado para as mensagens.....21

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

5 FOMRULÁRIOS COM DJANGO 5
5.1 Acessando o sistema 5
5.2 Criando o App de Usuários 7
5.3 Páginas para um usuário 9
5.4 Cadastrando o usuário 13
5.5 Autenticando o usuário 16
5.6 Melhorando a experiência do usuário 17
REFERÊNCIAS 22

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 FOMRULÁRIOS COM DJANGO

5.1 Acessando o sistema

Com as principais funcionalidades do site construídas, devemos limitar o acesso de quem utilizará os recursos que serão compartilhados entre os usuários. Em nosso caso, a biblioteca. O Django já disponibiliza um cadastro de usuários do sistema, mas ele é acessível apenas pela parte administrativa, conforme apresentado na Figura “Usuários para autenticação”.

Figura 5.1 – Usuários para autenticação
Fonte: Elaborado pelo autor (2020)

Através dessa funcionalidade, podem-se cadastrar diversos tipos de usuário. Vamos acessar o botão adicionar para verificar as opções que temos por meio desse site. Confira as opções na Figura “Cadastro de formulário do site administrativo”.

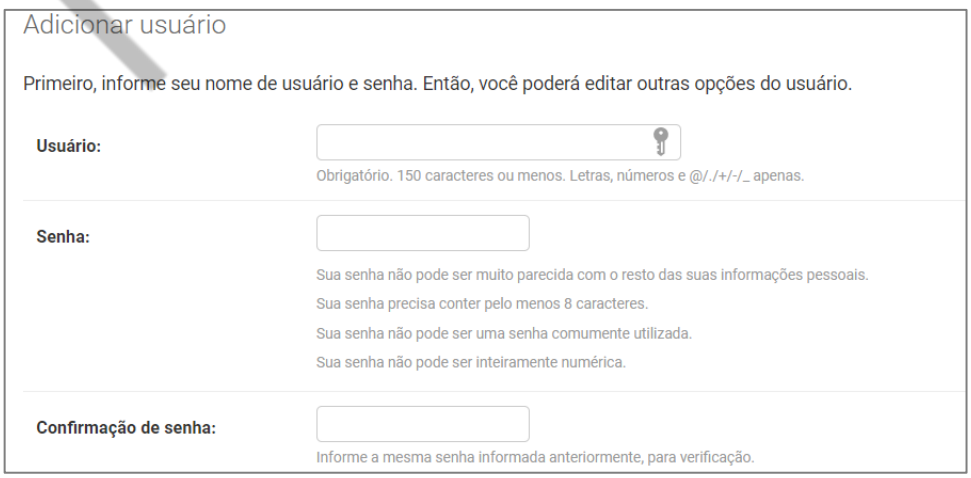

Figura 5.2 – Cadastro de formulário do site administrativo
Fonte: Elaborado pelo autor (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No formulário exibido na Figura “Cadastro de formulário do site administrativo”, encontramos a opção de cadastrar um nome de usuário e uma senha com o campo de confirmação. Após adicionar esses campos, é possível adicionar informações ao perfil como primeiro e último nome, endereço de e-mail, grupo ao qual ele pertence e as permissões de acesso. Veja na Figura “Opções de criação do perfil”.

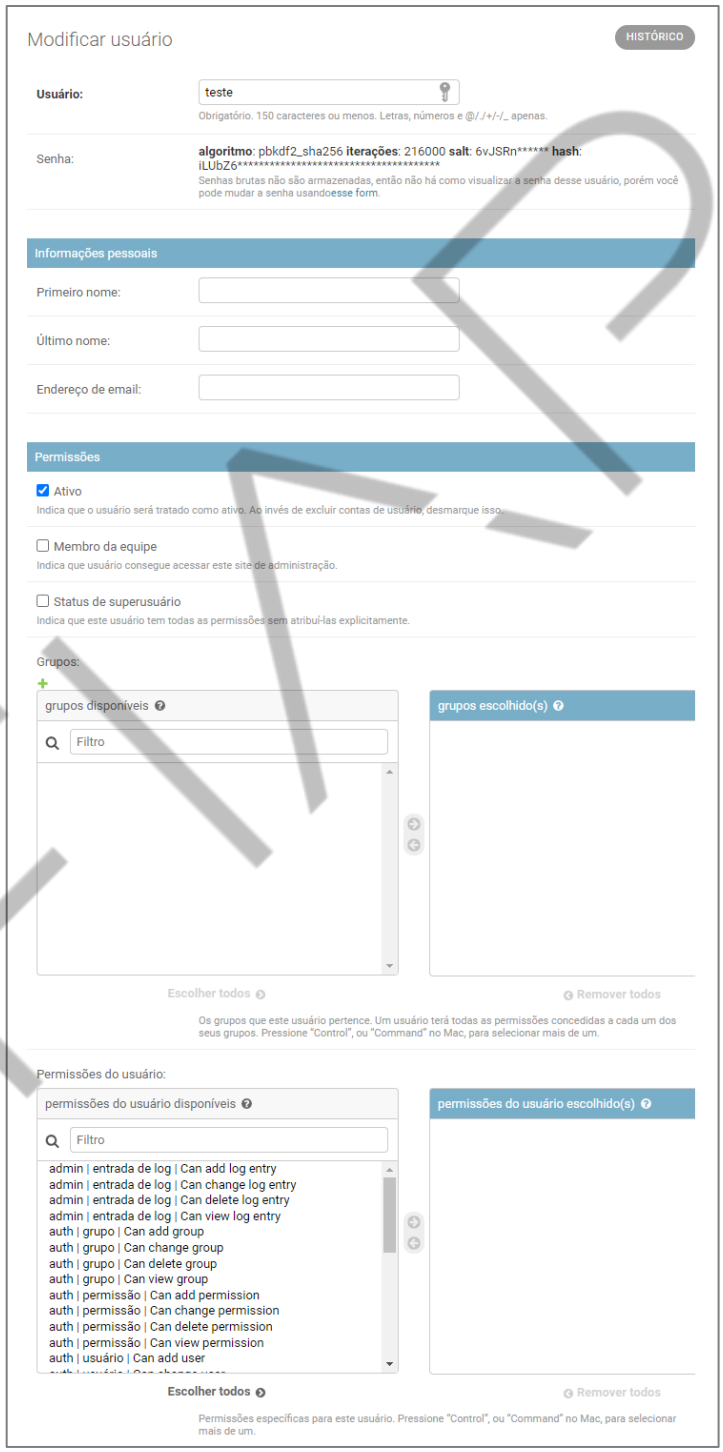

Figura 5.3 – Opções de criação do perfil
Fonte: Elaborado pelo autor (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Porém é inviável para o sistema que a criação de usuários seja realizada apenas pela parte administrativa. Precisamos criar um formulário para que a criação de acesso seja realizada através do nosso cliente. Para isso, vamos criar um app de usuários que controlará as rotas e os recursos necessários para cadastro e autenticação dos usuários no sistema.

5.2 Criando o App de Usuários

Para começar a configuração de acesso dos usuários ao nosso sistema de biblioteca, vamos criar um app específico para controlar as rotas e funções de usuários, como cadastro, login e logout. A criação do app é feita no terminal através do comando `python manage.py startapp usuarios`. Na sequência, devemos vincular ao nosso aplicativo principal, o library.

Acesse o arquivo settings.py e altere as opções de INSTALLED_APPS, conforme apresentado no Código-fonte “Alterando ExibeLivro para permitir edição rápida”.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'livros',
    'autores',
    'usuarios',
]
```

Código-fonte 5.1 – Alterando ExibeLivro para permitir edição rápida
Fonte: Elaborado pelo autor (2020)

Precisamos agora formar as rotas de acesso. No app usuários, vamos criar um arquivo urls.py dentro da pasta usuários, a que foi criada após o comando startapp.

O Código-fonte “Rotas de Usuário” apresenta a criação das rotas para cada umas das funcionalidades referentes a usuários que teremos em nosso sistema.

```
from django.urls import path
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from . import views

urlpatterns = [
    path('cadastro', views.cadastro, name='cadastro'),
    path('login', views.login, name='login'),
    path('logout', views.logout, name='logout'),
]
```

Código-fonte 5.2 – Rotas de Usuário
Fonte: Elaborado pelo autor (2020)

Para deixarmos o código pronto em questão de infraestrutura, pelo menos para iniciar a implementação do app de usuários, vamos inserir o Código-fonte “Estrutura principal do arquivo views.py do app usuarios” no arquivo views.py. Assim, a estrutura principal já estará pronta. Precisaremos apenas preencher o recheio do código.

```
from django.shortcuts import render

def cadastro(request):
    pass

def login(request):
    pass

def logout(request):
    pass
```

Código-fonte 5.3 – Estrutura principal do arquivo views.py do app usuarios
Fonte: Elaborador pelo autor (2020)

O próximo passo é registrar como serão chamadas as urls no navegador, para isso temos que registrar as informações no arquivo urls.py no app principal, o library. O Código-fonte “Vínculo das urls no aplicativo principal” apresenta a alteração necessária para que isso seja concluído.

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('', include('livros.urls')),
    path('usuarios/', include('usuarios.urls')),
    path('admin/', admin.site.urls),
] + static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

Código-fonte 5.4 – Vínculo das urls no aplicativo principal
Fonte: Elaborado pelo autor (2020)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Todos os templates estão dentro do app de livros. Isso ocorreu anteriormente porque só tínhamos ele, então fazia sentido manter todos os templates nele. No entanto a aplicação está crescendo e não é uma boa ideia manter todos os templates dentro desse app. A pasta de "templates" deve agora estar no mesmo nível dos apps. A partir dessa alteração, sempre que quisermos alterar um template ou alguma página html, não teremos a necessidade de lembrar em qual app devemos entrar. Moveremos a pasta de "livros" e deixaremos no mesmo nível dos apps.

Como movemos a pasta de templates, temos a necessidade de ir até o arquivo settings.py para ajustar as configurações da aplicação e informar o novo local. Em settings.py de library, temos um diretório dizendo onde estarão os arquivos templates — páginas html da aplicação. Como não estão mais em livros e sim no mesmo nível dos demais diretórios de nosso projeto, deixaremos apenas a palavra templates nas configurações, não mais livros/templates. Após essa alteração, nossa aplicação deve estar funcionando novamente.

Agora, devemos criar os templates de nossos formulários. É o que veremos na sequência.

5.3 Páginas para um usuário

Vamos começar criando a página de cadastro do usuário, mas não vamos deixar esses templates com os demais. Para mantermos a organização de nossa aplicação, vamos criar dentro de uma pasta chamada usuarios. Assim, todos os templates referentes a essa funcionalidade ficarão concentrados em um único lugar. O arquivo cadastro.html é apresentado no Código-fonte “Template do formulário de cadastro”.

```
{% extends 'base.html' %}
{% load static %}
{% block conteudo %}
    <div class="navbar navbar-expand-lg navbar-light bg-
light">
        {% include 'partials/_cabecalho.html' %}
    </div>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    <main class="container">
        <div class="col-12">
            <h2 class="mt-4 mb-5">Cadastrar nova conta</h2>
            <form>
                <div class="form-row">
                    <div class="form-group col-md-6">
                        <label for="nome">Nome</label>
                        <input type="text" class="form-
control" id="nome" name="nome" placeholder="ex: Andrey"
required />
                    </div>
                    <div class="form-group col-md-6">
                        <label
for="sobrenome">Sobrenome</label>
                        <input type="text" class="form-
control" id="sobrenome" name="sobrenome" placeholder="ex:
Masiero" required>
                    </div>
                </div>
                <div class="form-group">
                    <label for="email">E-mail</label>
                    <input type="email" class="form-control"
id="email" name="email" placeholder="ex:
andreymasiero@fiap.com" required>
                </div>
                <div class="form-row">
                    <div class="form-group col-md-6">
                        <label for="senha">Senha</label>
                        <input type="password" class="form-
control" id="senha" name="senha" placeholder="ex: abcd1234"
required />
                    </div>
                    <div class="form-group col-md-6">
                        <label for="confirmacao-
senha">Confirmação da senha</label>
                        <input type="password" class="form-
control" id="confirmacao-senha" name="confirmacao-senha"
placeholder="ex: abcd1234" required>
                    </div>
                </div>
                <button type="submit" class="btn btn-
primary">Criar sua conta</button>
            </form>
        </div>
    </main>
{% endblock %}
```

Código-fonte 5.5 – Template do formulário de cadastro
Fonte: Elaborado pelo autor (2020)

Na sequência, vamos criar o formulário de login através do arquivo login.html. Confira o conteúdo no decorrer do Código-fonte “Template do formulário de login”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
{% extends 'base.html' %}
{% load static %}
{% block conteudo %}
    <div class="navbar navbar-expand-lg navbar-light bg-
light">
        {% include 'partials/_cabecalho.html' %}
    </div>
    <main class="container">
        <div class="col-12">
            <h2 class="mt-4 mb-5">Login</h2>
            <form>
                <div class="form-group">
                    <label for="email">E-mail</label>
                    <input type="email" class="form-control"
id="email" name="email"
                           placeholder="ex:
andreymasiero@fiap.com" required>
                </div>
                <div class="form-group">
                    <label for="senha">Senha</label>
                    <input type="password" class="form-
control" id="senha" name="senha" placeholder="ex: abcd1234"
                           required/>
                </div>
                <div class="col-12 text-center">
                    <button class="btn btn-success"
type="submit">Acessar sua conta</button>
                    <button type="button" class="btn btn-
primary" onclick="irCadastro()">Ainda não tem conta?
                        Clique aqui
                    </button>
                </div>
            </form>
        </div>
    </main>
    <script>
        function irCadastro() {
            location.href = '{% url 'cadastro' %}';
        }
    </script>
{% endblock %}
```

Código-fonte 5.6 – Template do formulário de login
Fonte: Elaborado pelo autor (2020)

Para testarmos nossos templates, vamos alterar o arquivo views.py e chamar o método render em cada um dos métodos que vão trabalhar o comportamento de cada funcionalidade. Confira o Código-fonte “Renderização das telas”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
def cadastro(request):
    return render(request, 'usuarios/cadastro.html')


def login(request):
    return render(request, 'usuarios/login.html')
```

Código-fonte 5.7 – Renderização das telas
Fonte: Elaborado pelo autor (2020)

A tela de login é apresentada na figura abaixo.

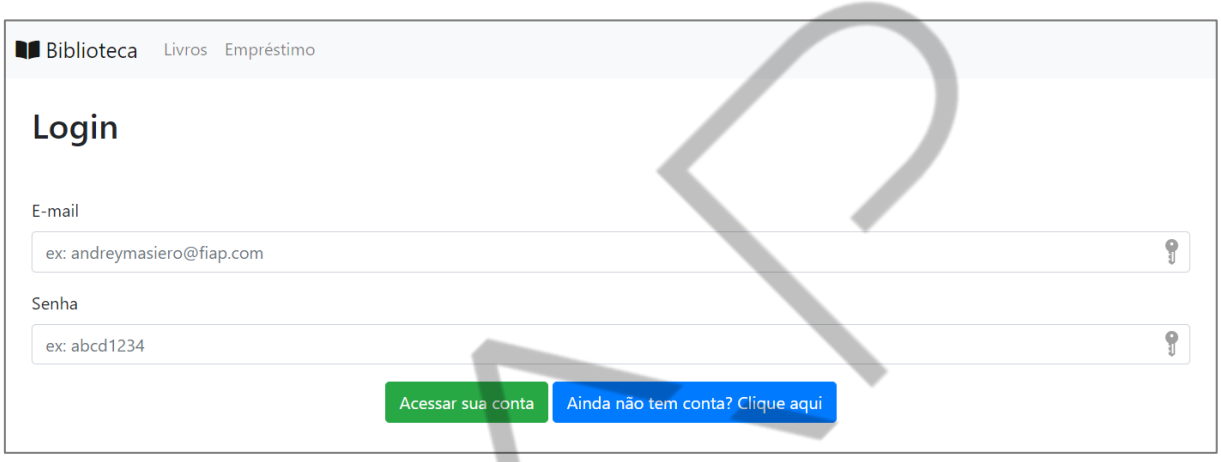

Figura 5.4 – Tela de login
Fonte: Elaborado pelo autor (2020)

Na figura a seguir, é apresentada a tela de cadastro do usuário.

Figura 5.5 – Tela de cadastro do usuário
Fonte: Elaborado pelo autor (2020)

Vamos ajustar o menu para habilitar o acesso ao login e ao cadastro do usuário. O partials de cabeçalho é apresentado no Código-fonte “Atualização do partial de cabeçalho”.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<header class="container">
    <a class="navbar-brand" href="{% url 'index' %}">
        <i class="fas fa-book-open"></i>
        Biblioteca
    </a>
    <button
            class="navbar-toggler"
            type="button"
            data-toggle="collapse"
            data-target="#navbarNavAltMarkup"
            aria-controls="navbarNavAltMarkup"
            aria-expanded="false"
            aria-label="Toggle navigation"
    >
        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
        <div class="navbar-nav">
            <a class="nav-link" href="{% url 'index'
%}">Livros</a>
            <a class="nav-link" href="{% url 'emprestimo'
%}">Empréstimo</a>
            <a class="nav-link" href="{% url 'cadastro'
%}">Crie sua conta</a>
            <a class="nav-link" href="{% url 'login'
%}">Login</a>
        </div>
    </div>
</header>
```

Código-fonte 5.8 – Atualização do partial de cabeçalho
Fonte: Elaborado pelo autor (2020)

Chegou o momento de trabalhar com a requisição de cadastro de um novo usuário.

5.4 Cadastrando o usuário

Visualizamos nossos formulários e o próximo passo é o de preencher os campos e clicar no botão "Criar sua conta" para que os dados sejam enviados e salvos na tabela auth_user, criando o usuário. Essa é a tabela em que o Django armazena os usuários, que simulamos a criação na parte de administração do sistema. A partir desse momento, conseguiremos realizar o login no site e acessar as opções de empréstimo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A grande questão é: como pegar as informações inseridas no formulário para criar o usuário? Antes de respondermos a essa questão, vale lembrar que o Django trabalha apenas com os verbos do HTTP get e post. Sendo assim, o método post é o mais adequado para trabalharmos com envio de informações que serão inseridas em nosso banco de dados. No caso de qualquer requisição que vá criar recursos em nosso sistema, é recomendável que utilizemos o POST para isso.

Então, para recuperar ou processar informações que estão num formulário, usaremos o verbo post do HTTP. Acessando o arquivo cadastro.html, dirija-se à linha 10, na qual há uma tag form. Inclua o atributo method="POST" e, na sequência, precisamos definir a ação que o formulário realizará ao criarmos a intenção de submetê-lo. Para vincular essa ação, adicionamos o atributo action, colocando a url para o método de cadastro, veja action="{% url 'cadastro' %}".

Por fim, vamos alterar o método cadastro no arquivo views.py do app de usuários. O Código-fonte "Recebendo a chamada do formulário" apresenta o código esperado.

```
def cadastro(request):
    if request.method == 'POST':
        print('Usuário cadastrado com sucesso')
        return redirect('login')
    else:
        return render(request, 'usuarios/cadastro.html')
```

Código-fonte 5.9 – Recebendo a chamada do formulário
Fonte: Elaborado pelo autor (2020)

Caso o verbo HTTP seja o post, exibiremos uma mensagem de usuário cadastrado com sucesso e redirecionaremos para a página de login, na qual o usuário poderá realizar o acesso; caso contrário, ele permanecerá na tela de cadastro. O resultado do teste é apresentado na Figura "Erro de CSRF".

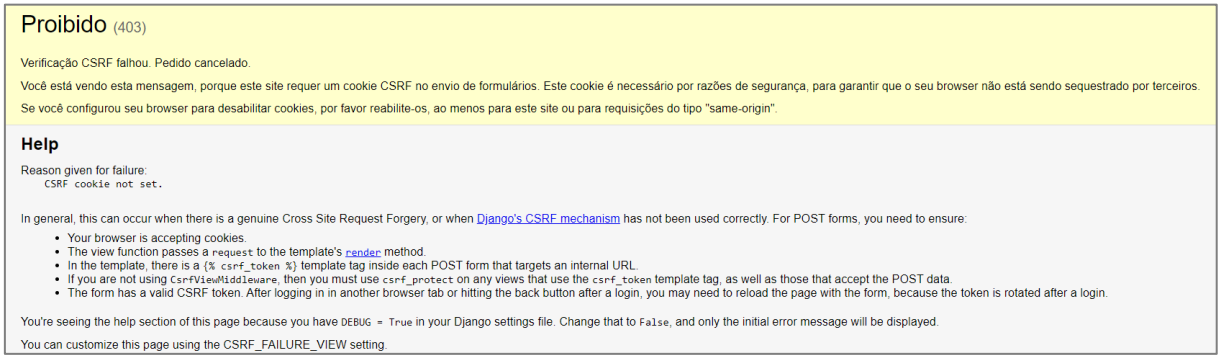

Figura 5.6 – Erro de CSRF
Fonte: Elaborado pelo autor (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ocorreu um erro de CSRF, mas o que significa isso? Ou seja, cross-site request forgery que, em português, significa falsificação de solicitação entre sites. Popularmente conhecido como "ataque de um clique". O Django já fornece proteção contra esse tipo de ataque.

Para evitarmos esse tipo de problema, utilizaremos um middleware, que tem acesso através de uma tag especial. Devemos, em nosso formulário, inserir crsf_token na nossa aplicação para enviar o token necessário. Sendo assim, em cadastro.html em que teremos a tag form, logo na sequência, incluiremos o token do csrf com o seguinte comando {% crsf_token %}. Nesse momento, é permitido enviar os dados ao servidor.

Agora, com o problema do CRSF resolvido, vamos utilizar o modelo para enviar as informações ao banco de dados e aproveitaremos para fazer as validações dos campos. O Código-fonte “Salvando o usuário” apresenta as alterações do método cadastro.

```
#! encoding: utf-8
from django.shortcuts import render, redirect
from django.contrib.auth.models import User


def cadastro(request):
    if request.method == 'POST':

        # Recebe os dados do formulário
        nome = request.POST['nome']
        sobrenome = request.POST['sobrenome']
        email = request.POST['email']
        senha = request.POST['senha']
        confirmacao_senha = request.POST['confirmacao-senha']

        # Validação dos campos
        if not nome.strip():
            print('Nome não pode ficar em branco')
            return redirect('cadastro')
        if not sobrenome.strip():
            print('Sobrenome não pode ficar em branco')
            return redirect('cadastro')
        if not email.strip():
            print('E-mail não pode ficar em branco')
            return redirect('cadastro')
        if senha != confirmacao_senha:
            print('Senha diferentes. Informe as mesmas
senhas, por favor.')
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            return redirect('cadastro')

        # Execução da chamada ao banco de dados
        User.objects.create_user(first_name=nome,
last_name=sobrenome, email=email,
                                 username=f'{nome}
{sobrenome}', password=senha)
        print('Usuário cadastrado com sucesso')
        return redirect('login')
    else:
        return render(request, 'usuarios/cadastro.html')
```

Código-fonte 5.10 – Salvando o usuário
Fonte: Elaborado pelo autor (2020)

Ao tentarmos cadastrar um usuário, ele será salvo em nossa base de dados. Podemos, então, conferir o sucesso da operação consultando os usuários cadastrados no sistema administrativo. Certifique-se de acordo com a Figura “Tabela de usuários cadastrados no sistema administrativo”.

Figura 5.7 – Tabela de usuários cadastrados no sistema administrativo
Fonte: Elaborado pelo autor (2020)

O próximo passo é efetuar o login do usuário.

5.5 Autenticando o usuário

Faremos o mesmo processo agora com a parte de login. Trabalhando primeiro no formulário, como post, e depois adicionando a tag do CSRF Token. Na sequência, vamos adicionar o Código-fonte “Método login”.

```
def login(request):
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    if request.method == 'POST':

        # Recebe os dados do formulário
        email = request.POST['email']
        senha = request.POST['senha']

        # Validação dos campos
        if email == '' or senha == '':
            print('E-mail e senha devem ser preenchidos')
            return redirect('login')

        # Autenticado usuário
        if User.objects.filter(email=email).exists():
            nome =
User.objects.filter(email=email).values_list('username',
flat=True).get()
            user = auth.authenticate(request, username=nome,
password=senha)
            if user is not None:
                auth.login(request, user)
                print('Login realizado com sucesso')
                return redirect('index')

    return render(request, 'usuarios/login.html')
```

Código-fonte 5.11 – Método login
Fonte: Elaborado pelo autor (2020)

Pronto, nosso login está funcionando com sucesso. Vamos, portanto, fazer algumas pequenas melhorias no sistema.

5.6 Melhorando a experiência do usuário

A primeira melhoria que faremos é em relação ao cabeçalho, vamos adicionar uma mensagem de boas-vindas com o nome do usuário e mostrar menus alternados de acordo com a autenticação dele. O Código-fonte “Melhoria na partial _cabecalho.html” apresenta a alteração na partial de cabeçalho.

```
<header class="container">
    <a class="navbar-brand" href="{% url 'index' %}">
        <i class="fas fa-book-open"></i>
        Biblioteca
    </a>
    <button
            class="navbar-toggler"
            type="button"
            data-toggle="collapse"
            data-target="#navbarNavAltMarkup"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            aria-controls="navbarNavAltMarkup"
            aria-expanded="false"
            aria-label="Toggle navigation"
    >
        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
        <div class="navbar-nav">
            {% if user.is_authenticated %}
            <a class="nav-link" href="{% url 'index'
%}">Livros</a>
            <a class="nav-link" href="{% url 'emprestimo'
%}">Empréstimo</a>
            <a class="nav-link" href="{% url 'logout'
%}">Sair</a>
            {% else %}
            <a class="nav-link" href="{% url 'cadastro'
%}">Crie sua conta</a>
            <a class="nav-link" href="{% url 'login'
%}">Login</a>
            {% endif %}
        </div>
    </div>
    {% if user.is_authenticated %}
    <p>Bem-vindo {{ user.username }}!</p>
    {% endif %}
</header>
```

Código-fonte 5.12 – Melhoria na partial _cabecalho.html
Fonte: Elaborado pelo autor (2020)

Na sequência vamos implementar o método de logout. Confira no Código-fonte “Método logout” como ficou a implementação.

```
def logout(request):
    auth.logout(request)
    return redirect('index')
```

Código-fonte 5.13 – Método logout
Fonte: Elaborado pelo autor (2020)

O próximo ponto é validar se o usuário está autenticado para acessar a rota de empréstimo. Para isso, vamos alterar o método de empréstimo no app de livros. Confira o Código-fonte “Validação de autenticação em uma rota”.

```
def emprestimo(request):
    if not request.user.is_authenticated:
        return redirect('index')

    dados = {
        'livros': Livro.objects.filter(emprestado=False)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    }
    print(dados)
    return render(request, 'index.html', dados)
```

Código-fonte 5.14 – Validação de autenticação em uma rota
Fonte: Elaborado pelo autor (2020)

Por último, vamos trabalhar com as mensagens de feedback ao usuário. O Django trabalha com níveis de mensagens e as message tags, que podem ser exploradas em detalhes na documentação do framework. Com esse sistema, é possível indicar, por exemplo, que uma mensagem é de erro e dizer qual é a message tag. Utilizamos o bootstrap na criação do layout de nossa aplicação, então conseguiremos utilizar tags do bootstrap para criar os alertas. Ele já possui componentes para as formas de exibir as mensagens de erro e de sucesso.

A diferença entre as mensagens de erro ou de sucesso será a classe renderizada no sistema. Na primeira, a classe utilizada será danger (que apresenta uma cor vermelha) e, na segunda, success (que apresenta a cor verde). Vamos criar nossas próprias message tags, deixando-as compatíveis com a documentação das classes do bootstrap. Acesse o arquivo settings.py de nosso app principal library, nele estão as configurações.

Em MIDDLEWARE, já teremos messages assim como em TEMPLATES há messages.context_processors, esses são recursos que podemos utilizar. No final do código, vamos construir nossas message tags para personalizarmos a aplicação: uma delas será para sucesso e a outra para erro. A alteração na configuração do arquivo settings.py é apresentada no Código-fonte “Recebendo a chamada do formulário”.

```
from django.contrib.messages import constants as messages

MESSAGE_TAGS = {
    messages.ERROR: 'danger',
    messages.SUCCESS: 'success',
}
```

Código-fonte 5.15 – Recebendo a chamada do formulário
Fonte: Elaborado pelo autor (2020)

Precisamos criar um template html para exibir as mensagens. Como pode ser aplicado em vários momentos no código, vamos criá-lo como uma nova partial chamada _alertas.html. No arquivo, em primeiro lugar, verificaremos se há alguma mensagem com {% if messages %} e fecharemos com {% endif %}. Caso haja alguma

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

mensagem, faremos um for para cada uma ser exibida. Veja como o Código-fonte “Recebendo a chamada do formulário” apresenta o código da partial.

```
{% if messages %}
    {% for message in messages %}
        <div class="alert alert-{{ message.tags }}"
role="alert">
            {{ message }}
        </div>
    {% endfor %}
{% endif %}
```

Código-fonte 5.16 – Recebendo a chamada do formulário
Fonte: Elaborado pelo autor (2020)

Vamos incluir a partial em dois lugares, primeiro no login. Em login.html, vamos inserir, antes do nosso formulário, o código {% include 'partials/_alertas.html' %}. Adicionaremos a partial também no cadastro, no mesmo lugar, antes do formulário. Vamos atualizar o método de cadastro para enviar as mensagens. Confira o Código--fonte “Método cadastro atualizado para as mensagens”.

```
from django.contrib import auth, messages


def cadastro(request):
    if request.method == 'POST':

        # Recebe os dados do formulário
        nome = request.POST['nome']
        sobrenome = request.POST['sobrenome']
        email = request.POST['email']
        senha = request.POST['senha']
        confirmacao_senha = request.POST['confirmacao-senha']

        # Validação dos campos
        if not nome.strip():
            messages.error(request, 'Nome não pode ficar em
branco')
            return redirect('cadastro')
        if not sobrenome.strip():
            messages.error(request, 'Sobrenome não pode ficar
em branco')
            return redirect('cadastro')
        if not email.strip():
            messages.error(request, 'E-mail não pode ficar em
branco')
            return redirect('cadastro')
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        if senha != confirmacao_senha:
            messages.error(request, 'Senha diferentes.
Informe as mesmas senhas, por favor.')
            return redirect('cadastro')

        # Execução da chamada ao banco de dados
        User.objects.create_user(first_name=nome,
last_name=sobrenome, email=email,
                                 username=f'{nome}
{sobrenome}', password=senha)
        messages.success(request, 'Usuário cadastrado com
sucesso')
        return redirect('login')
    else:
        return render(request, 'usuarios/cadastro.html')
```

Código-fonte 5.17 – Método cadastro atualizado para as mensagens
Fonte: Elaborado pelo autor (2020)

Vamos testar: observe que ao tentar realizar um cadastro com erro, veja na Figura “Tabela de usuários cadastrados no sistema administrativo” que a mensagem de erro aparece no topo do formulário.

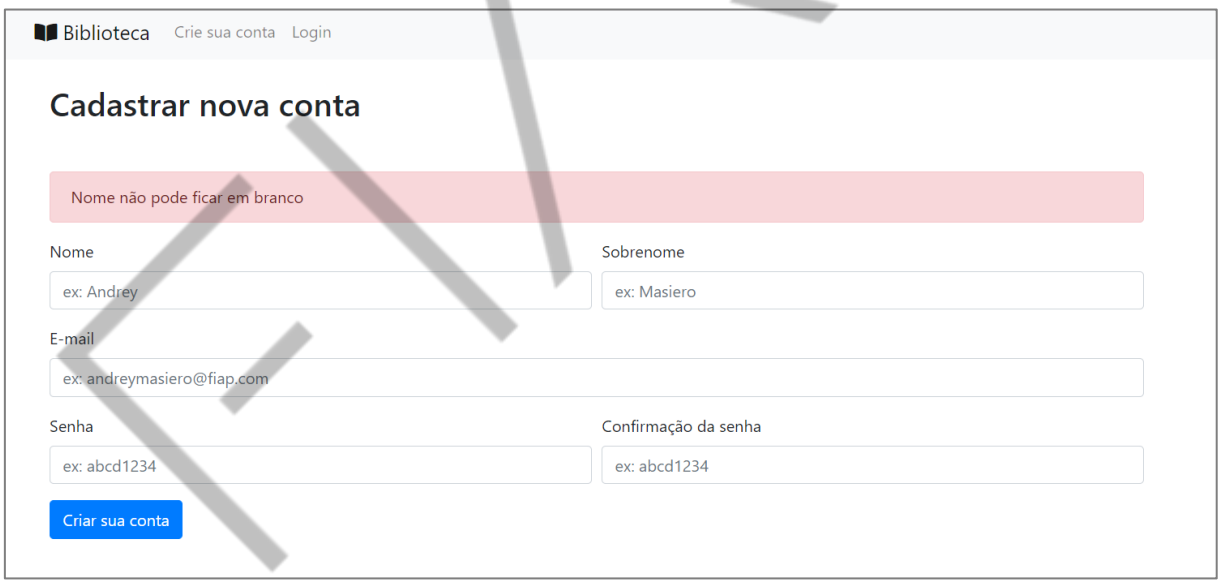

Figura 5.8 – Tabela de usuários cadastrados no sistema administrativo
Fonte: Elaborado pelo autor (2020)

Pronto, você já sabe toda a parte básica de como construir um sistema, utilizando o Django. Agora é com você! Vamos treinar cada vez mais para evoluirmos, um dia por vez, com nossas skills.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

Django. **Documentação do Django Web Framework**. [s.d.]. Disponível em: https://www.djangoproject.com/. Acesso em: 13 out. 2020.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com